

A HUMAN-LIKE AGENT FOR SCREEN-AWARE TASK AUTOMATION

Annesiyani Srikanthan

(IT22125248)

BSc (Hons) Degree in Information Technology
Specializing in Software Engineering

Department of Computer Science and Software Engineering

Sri Lanka Institute of Information Technology
Sri Lanka

August 2025

A HUMAN-LIKE AGENT FOR SCREEN-AWARE TASK AUTOMATION

Annesiyani Srikanthan

(IT22125248)

Dissertation submitted in partial fulfillment of the requirements for the Bachelor of
Science (Hons) in Information Technology Specializing in Software Engineering

Department of Computer Science and Software Engineering

Sri Lanka Institute of Information Technology
Sri Lanka

August 2025

DECLARATION

I declare that this is my own work and this dissertation¹ does not incorporate without acknowledgement any material previously submitted for a Degree or Diploma in any other University or institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Also, I hereby grant to Sri Lanka Institute of Information Technology, the nonexclusive right to reproduce and distribute my dissertation, in whole or in part in print, electronic or other medium. I retain the right to use this content in whole or part in future works (such as articles or books).

Signature

Date



25/04/2026

The above candidate has carried out research for the bachelor's degree Dissertation under my supervision.

Signature of the supervisor

Date



25/04/2026

ABSTRACT

This research presents the design and implementation of an Agentic AI Logic module for the Human-Like Agent for Screen-Aware Task Automation system. The primary objective of this component is to enable intelligent decision-making and autonomous task execution by orchestrating a closed-loop interaction cycle consisting of perception, planning, action, and evaluation.

The module functions as the central control unit of the system, integrating inputs from the visual perception component and structured action plans generated through language models. It interprets environmental feedback, maintains task context, and dynamically coordinates execution strategies to achieve user-defined goals. Through this process, the system continuously monitors task progress, validates outcomes, and adapts its behavior in response to incomplete actions, unexpected UI changes, or execution failures.

A key feature of the proposed approach is the implementation of a feedback-driven control loop, which allows the agent to iteratively refine its decisions based on real-time visual observations. This enables adaptive retry mechanisms, error handling, and exploratory interactions, thereby improving reliability in dynamic and unstructured graphical user interface (GUI) environments.

Unlike traditional automation frameworks that rely on static scripts or predefined workflows, the Agentic AI Logic module supports flexible and context-aware execution without requiring internal system access or application programming interfaces. By leveraging external visual feedback and intelligent coordination, the system is capable of operating effectively in black-box environments and legacy systems.

The integration of perception, decision-making, and actuation within a unified agentic framework demonstrates a scalable and extensible solution for autonomous GUI automation. Experimental evaluations in controlled scenarios indicate that the proposed module enhances robustness, adaptability, and human-like interaction capabilities in cross-platform testing contexts.

Keywords – Agentic AI, Autonomous Systems, GUI Automation, Feedback Loop, Black-Box Testing

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to all those who supported me throughout the successful completion of my final year research project.

First, I extend my heartfelt appreciation to my research supervisor, Mr. Vishan Jayasinghearachchi, for his continuous guidance, encouragement, and insightful feedback throughout this study. His support played a crucial role in shaping the direction and quality of this research.

I am also deeply thankful to my co-supervisor, Ms. Kaushalya Rajapaksa, for her valuable suggestions, constructive feedback, and support during the planning and development stages of this project.

Furthermore, I would like to acknowledge my research team members for their collaboration, support, and shared commitment in achieving our project goals.

Finally, I would like to express my deepest gratitude to my parents and friends for their constant encouragement, understanding, and unwavering support throughout this journey.

TABLE OF CONTENTS

DECLARATION	<i>i</i>
ABSTRACT	<i>ii</i>
ACKNOWLEDGEMENT.....	<i>iii</i>
LIST OF FIGURES	<i>vi</i>
LIST OF TABLES	<i>vii</i>
LIST OF ABBREVIATIONS	<i>viii</i>
1. INTRODUCTION.....	1
1.1 General Introduction	1
1.2 Background Literature	2
1.3 Limitation in the Current Approach	4
1.4 Research Gap	5
2. RESEARCH PROBLEM.....	7
2.1 Problem Context	7
2.2 Problem Statement	8
2.3 Core Research Problem.....	9
3. RESEARCH OBJECTIVES	10
3.1 Main Objective.....	10
3.2 Specific Objective	11
4. METHODOLOGY	12
4.1 System Architecture	12
4.2 Specific Component Description	17
4.2.1 Purpose	17
4.2.2 Workflow Overview	18
4.2.3 Input Processing.....	20
4.2.4 Context Structuring and Decision Preparation	21
4.2.5 Action Planning and Decision Generation	21
4.2.6 Perception-Driven Validation.....	21
4.2.7 Execution and Error Handling	22
4.2.8 Exploratory Branching.....	22
4.2.9 Optimization and Evaluation.....	23
4.3 Agentic Decision-Making and Action Planning Framework.....	24
4.3.1 Instruction Processing and Context Initialization	24
4.3.2 Context-Aware Reasoning Strategy	24
4.3.3 Action Decomposition and Planning.....	25
4.3.4 Structured Action Representation	25
4.3.5 Action Validation and Safety Checks.....	25

4.3.6	Feedback-Driven Adaptation	26
4.3.7	Continuous Optimization of Decision Strategy	26
4.4	Design System Diagrams.....	27
4.4.1	DFD Diagram	27
5.	IMPLEMENTATION AND TESTING	28
5.1	Implementation	28
5.1.1	Agentic AI Logic Overview.....	29
5.1.2	Instruction-to-Action Coordination Pipeline.....	30
5.1.3	Visual Context Construction and State Validation	31
5.1.4	Execution and HID Control Layer	31
5.1.5	Feedback Loop and Evaluation Mechanism.....	32
5.2	Testing	33
5.2.1	Test Plan and Strategy	33
5.2.2	Test Case Design	35
6.	RESULTS AND DISCUSSIONS.....	39
6.1	Results	39
6.1.1	Overall Execution Outcomes.....	40
6.1.2	Quantitative Performance Summary	45
6.1.3	Accuracy and Output Structure Quality	46
6.1.4	Robustness Under Edge Condition	48
6.1.5	Comparative Behavior Across Scenario Types	49
6.1.6	Practical Result Interpretation	50
6.2	Research Finding	51
6.3	Discussion	53
7.	CONCLUSIONS.....	58
8.	REFERENCES.....	61
9.	APPENDICES.....	63

LIST OF FIGURES

Figure 4.1: Overview of system architecture diagram	13
Figure 4.2: High-level layered architecture diagram	16
Figure 4.3: Agentic AI component diagram	18
Figure 4.4: Agentic AI processing workflow	20
Figure 4.5: DFD diagram for Agentic AI	27
Figure 5.1: Agentic AI endpoint flow diagram	29
Figure 5.2: Agentic instruction-to-action coordination flow.....	30
Figure 5.3: HID execution and retry mechanism flow	32
Figure 6.1: Agentic AI execution output (run JSON file).....	41

LIST OF TABLES

Table 1.1: Limitations and Proposed Directions.....	6
Table 4.1: Technologies, techniques, architecture and algorithms used.....	15
Table 5.1: Verify full login task execution using Agentic AI loop.....	36
Table 5.2: Verify visual perception and UI element detection.....	36
Table 5.3: Verify Agentic decision-making and task planning.....	37
Table 5.4: Verify HID execution and retry mechanism.....	38
Table 5.5: Verify feedback-driven evaluation and recovery logic.....	38

LIST OF ABBREVIATIONS

Abbreviation	Description
AI	Artificial Intelligence
LLM	Large Language Model
API	Application Programming Interface
UI	User Interface
GUI	Graphical User Interface
OCR	Optical Character Recognition
HID	Human Interface Device
RPA	Robotic Process Automation
VLM	Vision-Language Model
CV	Computer Vision
JSON	JavaScript Object Notation
HTTP	Hypertext Transfer Protocol
REST	Representational State Transfer
DFD	Data Flow Diagram
QA	Quality Assurance
QA	Quality Assurance Testing
ESP32	Low-cost microcontroller with Wi-Fi and Bluetooth capabilities
OCR	Optical Character Recognition
VLM	Vision-Language Model
Agentic AI	An AI system capable of autonomous decision-making through perception, planning, action, and feedback loops
Prompt Engineering	The practice of designing and optimizing inputs to guide LLM outputs effectively
Perceive–Plan–Act–Evaluate Loop	An iterative agent framework for continuous decision-making and execution in dynamic environments

1. INTRODUCTION

1.1 General Introduction

The Screen Aware Task Agent is an agentic artificial intelligence-based automation system designed to perform human-like interactions with graphical user interfaces (GUIs). Within this system, the Agentic AI Logic module serves as the central decision-making component, responsible for coordinating perception, planning, action, and evaluation processes to enable autonomous task execution. The system is particularly suited for environments where traditional automation techniques are ineffective, such as legacy systems, secure platforms, or applications without accessible APIs, as it relies on external visual observation and input emulation rather than internal system integration [1], [2].

The proposed system follows a closed-loop workflow consisting of perception, planning, action, and evaluation, where the Agentic AI Logic module orchestrates the entire process. During the perception stage, structured data representing the user interface is obtained from the visual perception component, which uses computer vision and optical character recognition (OCR) techniques to extract relevant screen information [3]. In the planning stage, structured action sequences generated through language models are interpreted and managed by the Agentic AI Logic module to align with the current task context [4]. The action stage involves executing these decisions through a hardware-based Human Interface Device (HID), while the evaluation stage relies on visual feedback and execution results to determine whether the intended outcome has been achieved.

A key responsibility of the Agentic AI Logic module is to maintain a continuous feedback-driven control loop. By analyzing real-time visual updates and execution responses, the module dynamically adapts its decisions, handles errors, and initiates retries when necessary. This enables the system to function reliably in dynamic and unpredictable GUI environments, where interface states may change frequently or deviate from expected behavior.

The system architecture follows a microservices-based design, where independent components such as vision processing, language model reasoning, and HID actuation operate as separate services. Within this architecture, the Agentic AI Logic module acts as the central orchestrator, managing communication between services and ensuring synchronized execution of tasks. This design supports scalability, modularity, and maintainability in line with established software engineering principles [5].

By integrating intelligent decision-making with continuous feedback and system coordination, the Agentic AI Logic module enables adaptive and context-aware automation. Unlike conventional rule-based approaches, this method supports flexible execution strategies and improves robustness in black-box testing scenarios. The implementation demonstrates the potential of agentic systems in advancing autonomous GUI interaction and contributes to ongoing research in intelligent automation and human-computer interaction [6].

1.2 Background Literature

Graphical User Interface (GUI) automation has been extensively studied to improve software quality and reduce repetitive manual interaction. Traditional approaches, including WebDriver-based automation and rule-based Robotic Process Automation (RPA), rely heavily on access to internal application structures such as Document Object Model (DOM) metadata, application programming interfaces (APIs), or accessibility trees [7], [8]. While these methods achieve high accuracy and efficiency in controlled environments, their effectiveness significantly decreases in black-box scenarios where such internal access is restricted or unavailable [9].

To overcome these limitations, vision-based automation techniques have been introduced, focusing on interpreting user interfaces through visual data. Optical Character Recognition (OCR) methods, such as those implemented in Tesseract OCR, are widely used to extract textual information from screen captures or camera-based inputs [10]. Additionally, datasets like Rico have contributed to understanding UI structures and enabling data-driven interface modeling [11]. However, these approaches primarily address perception and lack

integrated mechanisms for high-level decision-making and coordinated multi-step task execution.

Recent advancements in Large Language Models (LLMs) have introduced new possibilities for intelligent automation. Models such as Gemini demonstrate strong capabilities in understanding natural language instructions and generating structured action plans through prompt engineering and reasoning techniques [12], [13]. Furthermore, agentic frameworks such as ReAct combine reasoning and action in iterative loops, enabling more dynamic interaction patterns [14]. Despite these advancements, many LLM-based systems operate in simulated or API-supported environments, limiting their applicability in real-world scenarios that require physical interaction and continuous environmental feedback [15].

Hardware-based interaction methods, particularly those utilizing Human Interface Device (HID) emulation through microcontrollers such as ESP32, provide a practical solution for executing actions in restricted environments [16], [17]. These methods enable non-intrusive interaction with target systems by simulating keyboard and mouse inputs. However, HID-based approaches typically lack adaptive intelligence and depend on external control mechanisms to guide execution.

Overall, existing literature demonstrates significant progress in perception, reasoning, and execution as independent domains. However, there is a clear gap in integrating these components into a unified, feedback-driven control system. In particular, limited attention has been given to the design of an intelligent orchestration layer capable of coordinating perception, planning, action, and evaluation in a continuous loop. This gap highlights the need for an Agentic AI Logic framework that enables adaptive decision-making, real-time feedback utilization, and robust task execution in dynamic black-box environments.

1.3 Limitation in the Current Approach

Although the proposed system demonstrates the feasibility of screen-aware automation, several limitations remain, particularly from the perspective of agentic coordination and decision-making.

First, the system heavily depends on the accuracy of the perception module. Variations in environmental conditions such as lighting, camera alignment, motion blur, and screen reflections can introduce noise into the extracted visual data. Since the Agentic AI Logic relies on this input for decision-making, inaccuracies at the perception stage may propagate through the entire execution loop, affecting overall task reliability.

Second, the planning process, which is influenced by LLM-generated outputs, introduces uncertainty due to its probabilistic nature. Even with structured prompting, the generated action sequences may be incomplete, ambiguous, or misaligned with the actual interface state. From an agentic perspective, this creates challenges in ensuring consistent and deterministic execution, especially in complex or dynamic scenarios.

Third, the current system exhibits limitations in real-time responsiveness. The multi-stage workflow involving perception, reasoning, orchestration, and actuation introduces cumulative latency. This delay affects the ability of the Agentic AI Logic to react promptly to environmental changes, thereby limiting its effectiveness in time-sensitive tasks.

Fourth, the execution process depends on the reliability of hardware-based HID communication. Issues such as connection instability, synchronization delays, and inconsistent input handling by the host system can disrupt the execution flow. These challenges require the Agentic AI Logic to incorporate more robust error handling and recovery mechanisms.

Fifth, although the system includes a feedback loop, its evaluation capabilities are still relatively basic. The current implementation primarily verifies task completion at a surface level and lacks deeper semantic understanding of whether the intended objective has been fully achieved. This limits the system's ability to perform complex validation and adaptive learning over time.

Finally, the system has been evaluated in controlled environments with a limited range of applications. As a result, the robustness of the Agentic AI Logic in handling diverse interfaces, varying resolutions, and real-world uncertainties remains to be fully validated.

1.4 Research Gap

Although recent advancements in automation, computer vision, and Large Language Models (LLMs) have significantly improved the capabilities of intelligent systems, a critical gap remains in the development of unified, agent-driven control mechanisms for real-world GUI automation. Existing approaches tend to address perception, reasoning, and execution as separate components, with limited focus on how these elements can be effectively integrated into a continuous, feedback-driven decision-making framework.

One major limitation is the absence of a centralized orchestration mechanism capable of coordinating multiple system components in real time. While perception modules can extract interface data and LLMs can generate action plans, there is insufficient research on how these outputs are managed, validated, and executed within a dynamic environment. Without an intelligent control layer, systems often rely on static pipelines, resulting in rigid and non-adaptive behavior during task execution [18], [20].

Another significant gap is the lack of robust feedback-driven control loops. Many existing automation systems follow a linear execution model, where actions are performed sequentially without continuous validation of their outcomes. In real-world GUI environments, interface states may change unpredictably due to dynamic content, delays, or user interface variations. The absence of iterative evaluation and re-planning mechanisms limits the system’s ability to detect failures, recover from errors, and adapt its strategy accordingly [19], [25].

Furthermore, there is a disconnect between high-level planning and low-level execution. While LLMs can generate abstract action sequences, these plans are not always aligned with real-time environmental conditions or execution constraints. Existing approaches often assume ideal conditions and do not sufficiently address how decisions should be

adjusted based on partial observability, noisy perception data, or unsuccessful action outcomes. This gap highlights the need for an intelligent agent capable of contextual reasoning and dynamic decision refinement.

In addition, current systems provide limited support for adaptive retry strategies and exploratory behavior. When an action fails, most automation frameworks either terminate execution or repeat the same action without modification. There is a lack of mechanisms for intelligently exploring alternative actions, modifying strategies, or learning from previous execution outcomes, which are essential for achieving human-like interaction capabilities.

Moreover, many existing solutions are evaluated in controlled or simulated environments, where system inputs are clean and predictable. In contrast, real-world black-box environments introduce challenges such as noisy visual input, inconsistent interface layouts, and limited feedback signals. These conditions require a more robust and adaptive control mechanism that can operate effectively under uncertainty.

Therefore, a clear research gap exists in the design and implementation of an Agentic AI Logic framework that can orchestrate perception, planning, action, and evaluation within a unified, closed-loop system. Such a framework must support real-time feedback integration, dynamic decision-making, adaptive retry mechanisms, and robust coordination between system components to enable reliable automation in complex and unpredictable GUI environments.

Table 1. 1: Limitations and Proposed Directions

Aspect	Current Limitation	Impact	Proposed Direction
System Coordination	Lack of centralized orchestration	Disjointed component interaction	Introduce agentic control layer
Feedback Handling	No continuous evaluation loop	Inability to detect failures	Implement closed-loop evaluation

Execution Strategy	Static, linear workflows	Poor adaptability in dynamic UIs	Enable dynamic decision-making
Error Recovery	Repetitive or no retry logic	Low robustness	Introduce adaptive retry strategies
Real-world Applicability	Tested in controlled environments	Limited generalization	Design for noisy, black-box environments

The proposed Screen-Aware Task Agent addresses this gap by introducing an Agentic AI Logic module that acts as the central orchestrator of the system. This module integrates perception outputs, planning decisions, and execution feedback within a continuous perceive–plan–act–evaluate loop. By enabling real-time decision adaptation, feedback-driven validation, and intelligent retry mechanisms, the system improves robustness, flexibility, and reliability in real-world, access-restricted environments.

2. RESEARCH PROBLEM

2.1 Problem Context

Agentic artificial intelligence has emerged as a promising approach for enabling autonomous systems to perform complex tasks through continuous interaction with their environment. In the context of GUI automation, this involves interpreting visual interface states, generating appropriate actions, executing them, and evaluating the outcomes in a coordinated manner. Unlike traditional rule-based automation, agentic systems aim to replicate human-like behavior by adapting dynamically to changing conditions and incomplete information.

In modern GUI environments, automation is particularly challenging due to the absence of stable internal structures, especially in black-box systems where APIs, DOM access, or accessibility data are unavailable. These environments are often dynamic, with changing layouts, variable content, and unpredictable system responses. As a result, automation

requires not only accurate perception and planning but also continuous decision-making, context management, and adaptive control throughout the execution process.

Although individual technologies such as computer vision, LLM-based planning, and HID-based execution have advanced significantly, they are often implemented as isolated components. The lack of an intelligent coordination mechanism limits their ability to function effectively as a unified system. Without a central agent to manage interactions between these components, automation processes remain static, fragile, and unable to respond effectively to real-time changes.

2.2 Problem Statement

Despite the availability of advanced perception, reasoning, and execution technologies, existing GUI automation systems lack a robust Agentic AI Logic framework capable of coordinating these components within a continuous, feedback-driven control loop.

One of the primary challenges is the absence of a centralized decision-making mechanism to manage the interaction between perception, planning, and execution stages. In many existing approaches, these components operate independently, resulting in disjointed workflows and limited adaptability during task execution. This makes it difficult to ensure that actions are consistently aligned with the current state of the user interface.

Another key issue is the lack of effective context management across multi-step tasks. GUI automation often involves sequential actions that depend on previous outcomes. However, without a mechanism to maintain and update task context, systems may lose track of progress, leading to incorrect or redundant actions and reducing overall reliability.

Furthermore, existing systems exhibit limited capability for real-time evaluation and feedback integration. Actions are typically executed without sufficient validation of their outcomes, and failures are not always detected or handled appropriately. This restricts the system's ability to adapt to unexpected interface changes, delays, or execution errors.

In addition, the absence of adaptive decision-making and retry strategies reduces the robustness of automation systems. When an action fails, most systems either terminate execution or repeat the same step without modification. This behavior is insufficient for dynamic environments, where alternative strategies and exploratory actions may be required to achieve the desired outcome.

Therefore, the core problem addressed in this research is the design and implementation of an Agentic AI Logic module that can enable continuous, context-aware, and feedback-driven decision-making for GUI automation. The proposed solution aims to bridge the gap between perception, planning, and execution by introducing a unified control mechanism that supports dynamic adaptation, error handling, and reliable task completion in real-world black-box environments.

2.3 Core Research Problem

Therefore, the core research problem addressed in this study is the absence of a robust and integrated **Agentic AI Logic framework** capable of orchestrating perception, planning, action, and evaluation within a unified, feedback-driven control loop for GUI automation.

While existing systems can interpret user instructions and generate action sequences, they lack a centralized decision-making mechanism to manage the end-to-end execution process. In particular, there is no effective approach for maintaining contextual awareness across multi-step tasks, validating execution outcomes in real time, and dynamically adapting decisions based on environmental feedback.

To address this gap, the system must incorporate a control framework that is capable of coordinating multiple components, including visual perception, action planning, and hardware-based execution, while ensuring consistency and reliability throughout the task lifecycle. This framework should support structured decision-making, continuous evaluation, and adaptive re-planning in response to failures, incomplete actions, or changes in the user interface.

Addressing this problem is essential for enabling reliable and scalable automation in complex, real-world environments where systems must operate under uncertainty, handle dynamic interface conditions, and function without internal system access. The development of such an agentic control mechanism represents a critical step toward achieving autonomous, human-like interaction in GUI-based applications.

3. RESEARCH OBJECTIVES

3.1 Main Objective

The primary objective of this study is to design and implement a robust **Agentic AI Logic framework** capable of enabling autonomous, context-aware GUI automation through a continuous perceive–plan–act–evaluate loop. The system aims to coordinate perception, planning, and execution components to achieve reliable and adaptive task completion in dynamic environments.

The proposed solution focuses on bridging the gap between high-level task intentions and real-world execution by introducing an intelligent control mechanism that can interpret system states, manage task context, and make informed decisions throughout the execution lifecycle. It is designed to support both deterministic functional tasks and adaptive exploratory interactions, enabling human-like behavior in complex and unpredictable interface conditions.

Furthermore, the system is intended to operate effectively in black-box environments without requiring internal application access, ensuring flexibility and applicability across diverse platforms. Ultimately, the objective is to develop a scalable and intelligent automation framework that can continuously monitor, evaluate, and refine its actions to achieve reliable task execution.

3.2 Specific Objective

The specific objectives of this study are as follows:

- To design and implement a **closed-loop agentic workflow** based on the Perceive–Plan–Act–Evaluate cycle, enabling continuous interaction between system components and the environment.
- To develop a **centralized decision-making module** that coordinates outputs from visual perception and planning components, ensuring that actions are contextually relevant and aligned with the current interface state.
- To implement **context management mechanisms** that maintain task state across multi-step interactions, enabling consistent and sequential execution of complex automation workflows.
- To design a **feedback-driven evaluation system** that verifies the outcome of each executed action using real-time visual input, ensuring that intended objectives are achieved.
- To develop **adaptive retry and decision refinement strategies** that allow the system to handle execution failures, unexpected interface changes, and incomplete actions through dynamic re-planning.
- To integrate the Agentic AI Logic with hardware-based execution (HID) and perception modules, ensuring synchronized and reliable end-to-end task execution.
- To enable **exploratory and flexible interaction capabilities**, allowing the system to adjust its behavior intelligently in dynamic and unpredictable GUI environments.
- To ensure **robust operation in black-box environments** by relying solely on external visual feedback and coordinated decision-making without requiring internal system access.

4. METHODOLOGY

The methodology of the Agentic AI Logic module in the Human-Like Agent for Screen-Aware Task Automation system is designed to enable intelligent, feedback-driven task execution through a closed-loop control mechanism. Unlike conventional approaches that focus primarily on instruction generation, this methodology emphasizes continuous decision-making, coordination, and validation across all stages of the automation process.

The system operates based on an agentic workflow consisting of four key stages: perception, planning, action, and evaluation. The Agentic AI Logic module acts as the central control unit that orchestrates these stages, ensuring that each action is aligned with the current interface state and overall task objective.

4.1 System Architecture

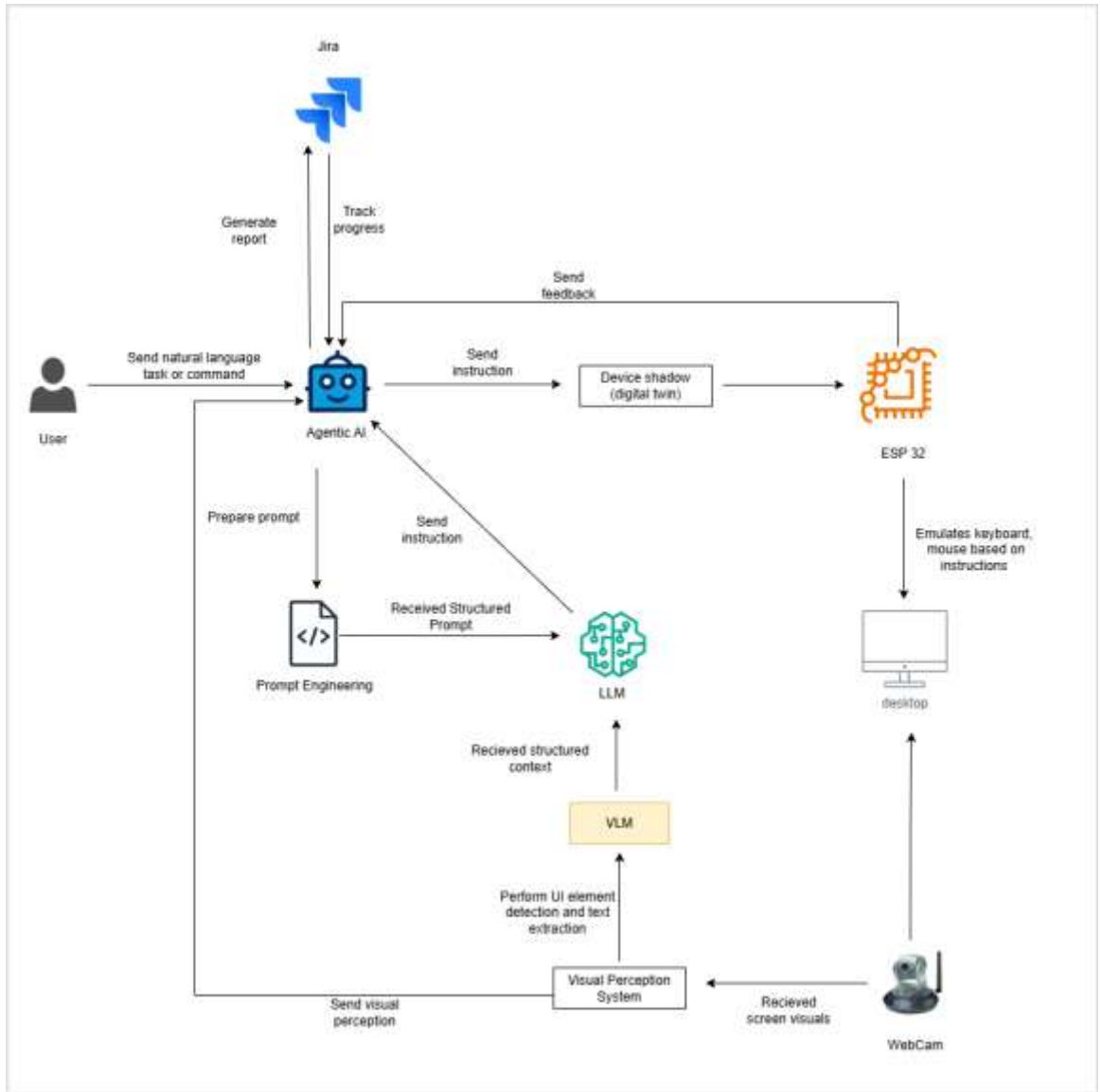


Figure 4. 1: Overview of system architecture diagram

Figure 4.1 illustrates the overall architecture of the Screen-Aware Task Agent, designed as a closed-loop, agentic automation system for black-box GUI environments. The architecture is built around external observation and hardware-level interaction, allowing the system to operate without relying on internal APIs, source code, or application-level hooks.

At the core of the architecture is the **Agentic AI Logic module**, which functions as the central decision-making and orchestration layer. The system begins with the user providing a high-level natural language instruction through the frontend interface.

This instruction is processed by the Agentic AI Logic, which maintains task context and initiates the execution cycle.

Rather than directly executing actions, the Agentic AI Logic coordinates with the planning component to obtain structured action steps. These plans are generated based on the current task context and visual perception data. The perception module continuously captures the target screen using an external camera and applies computer vision and OCR techniques to extract relevant UI information. This structured perception output is fed into the Agentic AI Logic, enabling it to make context-aware decisions.

Once a valid action is determined, the Agentic AI Logic forwards execution commands to the hardware interface layer. The system uses an ESP32-based Human Interface Device (HID) module to emulate keyboard and mouse inputs, allowing interaction with the target system in a non-intrusive and platform-independent manner.

A key feature of this architecture is the **feedback-driven evaluation loop**. After each action is executed, the system captures the updated screen state and re-evaluates the outcome. The Agentic AI Logic compares the expected result with the observed state and determines the next step. This may involve proceeding with the next action, retrying the current step, or generating a revised plan. This continuous validation mechanism enables the system to adapt dynamically to UI changes, delays, or unexpected behaviors.

In addition to execution, the architecture includes logging and monitoring capabilities. Action histories, decision states, and execution outcomes are recorded to support debugging, performance analysis, and reporting. Integration with external tools such as Jira enables traceability and enhances the system's applicability in real-world testing environments.

Table 4. 1: Technologies, techniques, architecture and algorithms used

Category	Description
Technologies	FastAPI, React, OCR (Tesseract), ESP32 HID, JSON-based communication
Techniques	Agentic decision-making, context management, feedback-driven evaluation, adaptive retry strategies
Algorithms	Perceive–Plan–Act–Evaluate loop, state validation, decision branching, retry and recovery logic
Architectures	Microservices architecture, agentic closed-loop system

Figure 4. 2: High-level layered architecture diagram

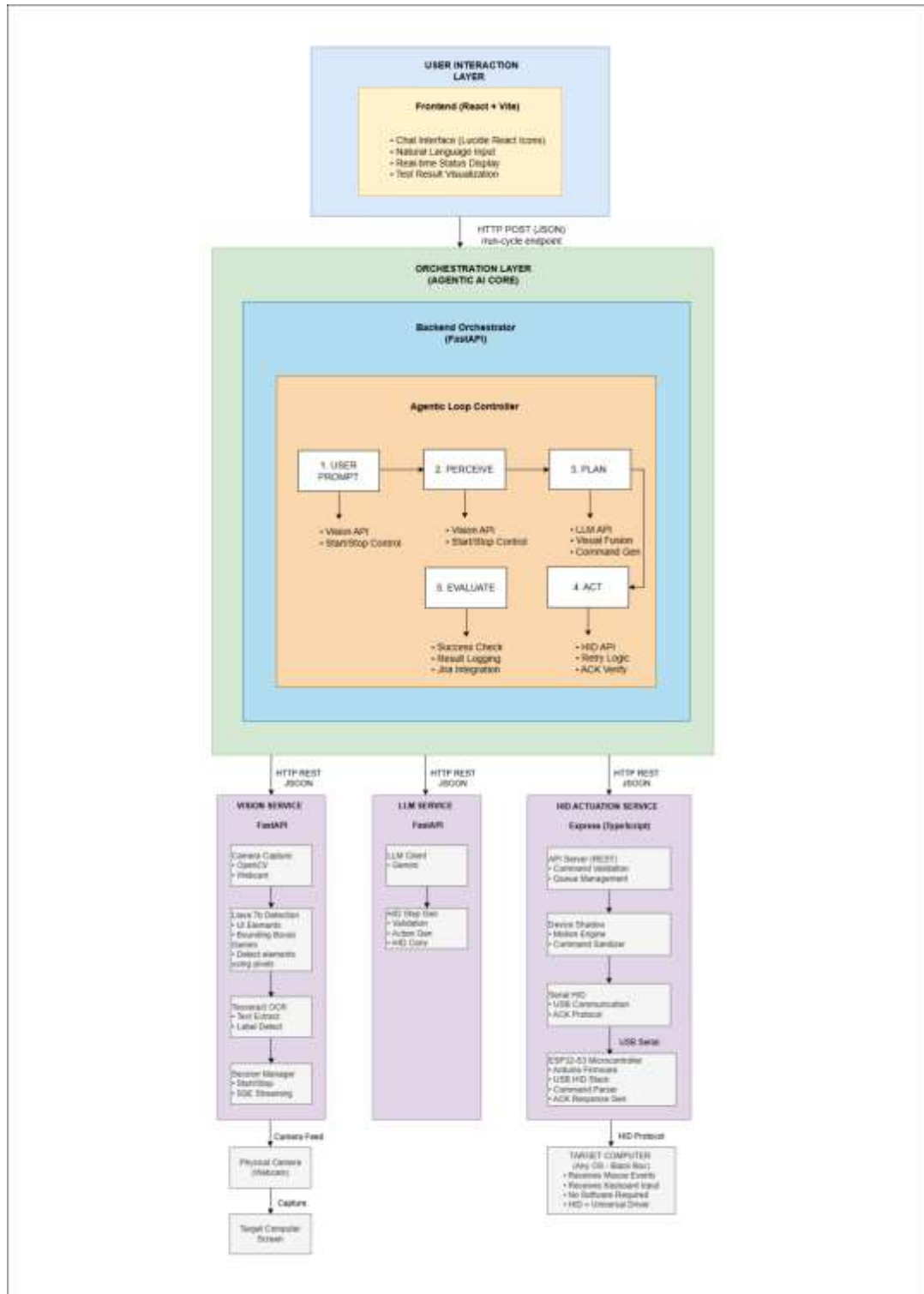


Figure 4.2 presents the layered architecture of the system, where each layer represents a functional component of the automation pipeline. The perception layer handles visual data acquisition and processing, the planning layer generates structured action sequences, and the execution layer performs physical interactions through HID. Overseeing all these layers is the Agentic AI Logic layer, which coordinates communication, manages context, and ensures consistent execution across the system.

4.2 Specific Component Description

The LLM and Prompt Engineering module is the central intelligence unit responsible for transforming high-level natural language instructions into structured, executable UI action plans for the Agentic AI Core. This component acts as the bridge between human intent and machine execution, ensuring that the system can understand, plan, and adapt in a human-like manner during testing.

4.2.1 Purpose

The LLM The purpose of the Agentic AI Logic module is to enable intelligent, context-aware, and feedback-driven control of the GUI automation process. It acts as the central decision-making unit that coordinates the entire task execution workflow, ensuring that high-level user objectives are achieved through reliable and adaptive system behavior.

Rather than solely interpreting instructions, the module manages the complete **Perceive–Plan–Act–Evaluate cycle**. It maintains task context, integrates visual perception data, and processes structured action plans to determine appropriate execution steps. By continuously monitoring the current UI state and comparing it with expected outcomes, the module ensures that each action is correctly aligned with the intended objective.

A key function of the Agentic AI Logic is to support **dynamic adaptation**. When discrepancies, failures, or unexpected interface changes are detected, the module evaluates the situation and adjusts its decisions accordingly. This includes triggering retries, modifying execution strategies, or initiating re-planning processes to maintain progress toward task completion.

Additionally, the module ensures that all actions are executed in a structured and controlled manner, enabling consistency and reliability across both functional and exploratory testing scenarios. Through continuous feedback integration and decision refinement, the Agentic AI Logic enhances system robustness and enables human-like interaction in dynamic, real-world environments.

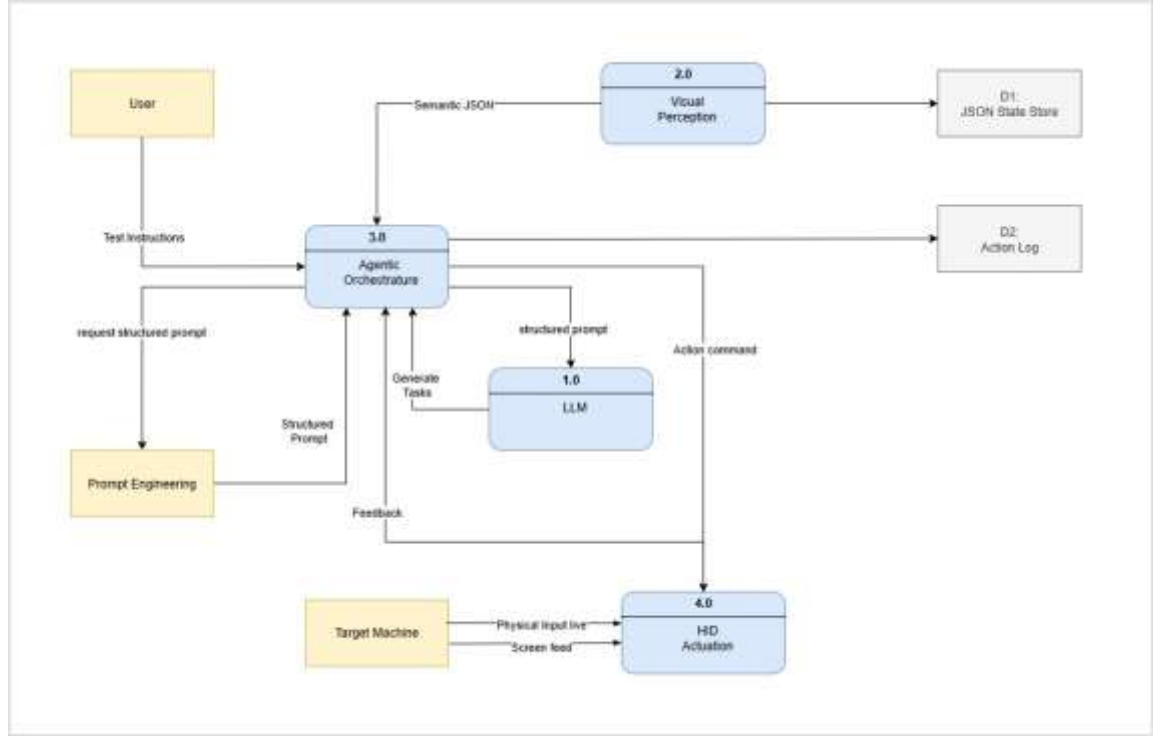


Figure 4. 3: Agentic AI component diagram

4.2.2 Workflow Overview

The Agentic AI Logic module operates through a structured, closed-loop workflow that transforms user instructions into validated and adaptive execution actions. This workflow is based on the **Perceive–Plan–Act–Evaluate cycle**, enabling continuous interaction between the system and the environment.

Instruction Intake and Task Initialization:

The process begins with the user providing a high-level task instruction (e.g., “Open the settings menu and reduce the volume to 50%”). The Agentic AI Logic module

initializes the task by storing the objective, defining execution boundaries, and preparing an internal task state. This step ensures that all subsequent decisions remain aligned with the overall goal.

Perception and Context Acquisition:

The system continuously captures the current screen state through the visual perception module. Using OCR and UI detection techniques, relevant interface elements and textual information are extracted and converted into structured data. The Agentic AI Logic consumes this perception output to maintain an up-to-date understanding of the environment.

Planning and Decision Making:

Based on the current task state and perceived UI context, the Agentic AI Logic determines the next action. It coordinates with the planning component (LLM) to generate structured action steps when required. These steps are evaluated and refined by the agent to ensure they are consistent with the current interface state and execution constraints.

Action Execution:

Once a valid decision is made, the Agentic AI Logic sends structured commands to the execution layer. These commands are translated into low-level input actions and executed through the ESP32-based HID module, enabling interaction with the target system as if performed by a human user.

Evaluation and Feedback Integration:

After each action, the system captures the updated screen state and performs evaluation. The Agentic AI Logic compares the expected outcome with the observed result to determine whether the action was successful. This validation step is critical to ensure correctness and prevent error propagation.

Adaptive Decision and Retry Mechanism:

If discrepancies or failures are detected, the Agentic AI Logic dynamically adjusts its strategy. This may include retrying the action, modifying parameters, or requesting a revised plan from the planning component. This adaptive behavior allows the system to handle unexpected UI changes and incomplete actions effectively.

Exploratory Interaction Support:

In exploratory scenarios, the Agentic AI Logic enables flexible decision-making by selecting alternative interaction paths based on the current UI state. Instead of strictly following predefined steps, the system can adaptively explore the interface, similar to a human tester, while still maintaining task objectives and constraints.



Figure 4. 4: Agentic AI processing workflow

4.2.3 Input Processing

Natural language commands are received through the system interface (e.g., “Open the settings menu and change the screen brightness to 70%”). The Agentic AI Logic module initializes task execution by normalizing the input, removing ambiguity, and

extracting the core intent. This step ensures that the instruction is consistently represented within the system's internal task state, enabling reliable downstream decision-making.

4.2.4 Context Structuring and Decision Preparation

The Agentic AI Logic module constructs a structured internal context representation by combining user instructions, current UI perception data, and execution constraints. This structured context serves as the basis for all subsequent decisions in the control loop.

Prompt templates are used as a supporting mechanism to interface with the LLM when reasoning assistance is required. These templates ensure that any generated output is aligned with the current system state and maintains a consistent structured format. Rather than relying solely on static prompting, the system dynamically updates context based on real-time feedback from the environment.

4.2.5 Action Planning and Decision Generation

The planning process is managed by the Agentic AI Logic module, which determines the next best action based on current task progress and perceived UI state. When required, the module invokes the LLM to assist in generating structured action sequences.

Complex tasks are decomposed into smaller executable steps, where each step represents a single interaction with the interface (e.g., click, type, select). However, all generated plans are validated and interpreted by the Agentic AI Logic before execution to ensure consistency with the actual UI state and task objective.

4.2.6 Perception-Driven Validation

The system continuously integrates outputs from the visual perception module to validate decision accuracy. UI elements detected through OCR and computer vision are mapped against expected targets in the action plan.

This creates a perception-driven validation loop where:

- perceived UI state influences planning decisions
- execution results influence future perception interpretation

The Agentic AI Logic ensures that all actions are grounded in the real-time interface state rather than assumed conditions.

4.2.7 Execution and Error Handling

Once an action is approved, the Agentic AI Logic module dispatches execution commands to the HID-based actuation layer. These commands are executed through keyboard and mouse emulation on the target system.

After execution, the system captures the updated UI state and evaluates whether the expected outcome has been achieved. If discrepancies or failures are detected, the Agentic AI Logic initiates recovery strategies such as:

- retrying the action
- adjusting parameters
- requesting a revised plan
- re-evaluating UI state before proceeding

This ensures robust operation in dynamic and unpredictable environments.

4.2.8 Exploratory Branching

In exploratory testing scenarios, the Agentic AI Logic module enables non-linear interaction behavior by allowing alternative action paths based on observed UI changes.

Instead of strictly following predefined sequences, the system can:

- explore alternative interface elements

- test multiple interaction paths
- adapt behavior based on unexpected UI states

This enables the system to simulate human-like exploratory testing behavior while still maintaining goal-oriented control.

4.2.9 Optimization and Evaluation

System performance is continuously evaluated using execution logs and feedback signals generated during task execution. The Agentic AI Logic module uses this feedback to refine decision strategies and improve future performance.

Evaluation is based on:

- task completion success rate
- correctness of action execution
- adaptability to UI changes
- effectiveness of recovery mechanisms
- reduction of failed or redundant actions

These metrics are used to assess the robustness of the agentic control loop in real-world black-box environments.

4.3 Agentic Decision-Making and Action Planning Framework

This section presents the methodology used by the Agentic AI Logic module for coordinating reasoning, planning, and execution in the proposed system. The objective is to enable reliable transformation of high-level user instructions into validated, structured UI actions while maintaining continuous awareness of dynamic interface states.

Although multiple language model backends can be integrated into the system, the LLM is used as a supporting reasoning component within the Agentic AI control loop. The primary responsibility of decision-making, validation, and execution coordination lies with the Agentic AI Logic module, which ensures that all actions remain consistent with real-time environmental feedback.

4.3.1 Instruction Processing and Context Initialization

User instructions are received and normalized by the Agentic AI Logic module to remove ambiguity and extract the core task objective. This instruction is then combined with contextual information, including current UI state from perception, previously executed actions, and system constraints.

This structured context becomes the foundation for all subsequent decisions in the agentic loop.

4.3.2 Context-Aware Reasoning Strategy

The Agentic AI Logic module constructs a structured reasoning context that guides decision-making. When required, the LLM is invoked as a reasoning assistant to generate possible action paths.

Prompt templates are used as a structured interface to ensure consistency in reasoning outputs. However, all outputs are interpreted and validated by the Agentic AI Logic before execution, ensuring that reasoning remains grounded in the actual UI state.

4.3.3 Action Decomposition and Planning

High-level objectives are decomposed into smaller, atomic UI actions by the Agentic AI Logic module, supported by LLM-generated suggestions when necessary. Each action represents a single interaction such as click, type, or selection.

The agent evaluates each proposed step against the current UI context to ensure feasibility and correctness before execution. This prevents invalid or outdated actions from being executed in dynamic environments.

4.3.4 Structured Action Representation

Validated actions are converted into a structured format (e.g., JSON-based representation) for execution. Each action includes:

- action type
- target UI element
- parameters
- expected outcome

This structured representation allows reliable communication between the decision layer and execution layer.

4.3.5 Action Validation and Safety Checks

Before execution, the Agentic AI Logic module performs validation checks to ensure:

- action consistency with UI state
- schema correctness
- task alignment

- feasibility of execution

If inconsistencies or invalid actions are detected, the system rejects or revises the plan using updated context.

4.3.6 Feedback-Driven Adaptation

After each action execution, the system captures updated UI state through the perception module. The Agentic AI Logic compares observed outcomes with expected results.

If mismatches are detected, the system initiates adaptive recovery mechanisms such as:

- re-planning the next step
- modifying execution strategy
- adjusting context state
- requesting alternative action paths

This ensures continuous correction and robustness in dynamic environments.

4.3.7 Continuous Optimization of Decision Strategy

The Agentic AI Logic module continuously improves its decision-making strategy using execution logs and failure patterns. Optimization focuses on:

- improving action accuracy
- reducing execution failures
- minimizing unnecessary re-planning
- increasing robustness in UI variability

This feedback-driven refinement enhances long-term system reliability in real-world black-box environments.

4.4 Design System Diagrams

System design diagrams were developed to identify the essential components and structure the implementation of the proposed model. Figure 4.4 illustrates the Data Flow Diagram (DFD), which represents the flow of data between the system components and supports a clear understanding of system interactions. This diagram was used as a reference during the design and implementation of the proposed research component.

4.4.1 DFD Diagram

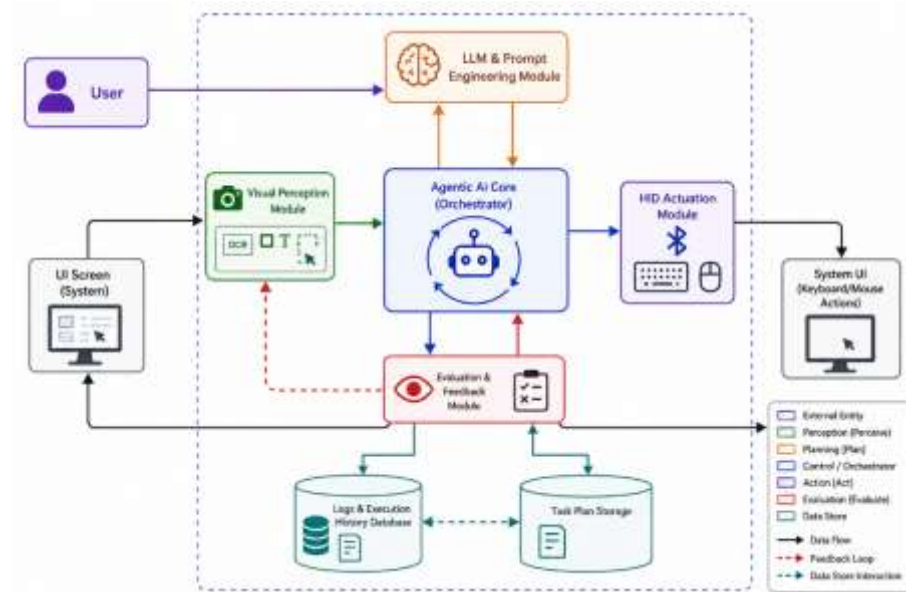


Figure 4. 5: DFD diagram for Agentic AI

5. IMPLEMENTATION AND TESTING

5.1 Implementation

The implementation focuses on an **agentic, screen-aware automation system** where the Agentic AI Logic module acts as the central control unit responsible for coordinating perception, planning, execution, and evaluation. The system is designed around a continuous **Perceive–Plan–Act–Evaluate loop**, enabling adaptive and context-aware GUI automation in black-box environments.

Instead of relying on static execution pipelines, the Agentic AI Logic maintains real-time task state and dynamically manages decision-making based on visual feedback. User instructions are interpreted, contextualized with perception data, and converted into structured execution strategies that are validated before being sent to the actuation layer.

The system integrates multiple components including visual perception, LLM-based reasoning support, and HID-based execution. However, all outputs from these components are controlled and validated by the Agentic AI Logic to ensure consistency, correctness, and alignment with the current UI state.

The implementation includes **feedback-driven adaptation**, where every executed action is verified using updated screen observations. Based on this evaluation, the Agentic AI Logic decides whether to proceed, retry, or re-plan the next action. This enables robust operation in dynamic and unpredictable interface conditions.

5.1.1 Agentic AI Logic Overview

The Agentic AI Logic module serves as the **central orchestration and decision-making engine** of the system. It is responsible for managing the complete lifecycle of task execution, from initial instruction intake to final task completion.

When a user instruction is received, the module initializes an internal task state and coordinates with perception and reasoning components to understand the current environment. It then determines the next best action based on the current UI state and task objective.

Although the LLM microservice is used as a reasoning support component, all generated outputs are treated as **candidate action proposals** rather than final execution instructions. The Agentic AI Logic module validates, refines, and approves these actions before execution.

This design ensures separation between reasoning and control, allowing the system to remain stable, predictable, and adaptable across different GUI environments.

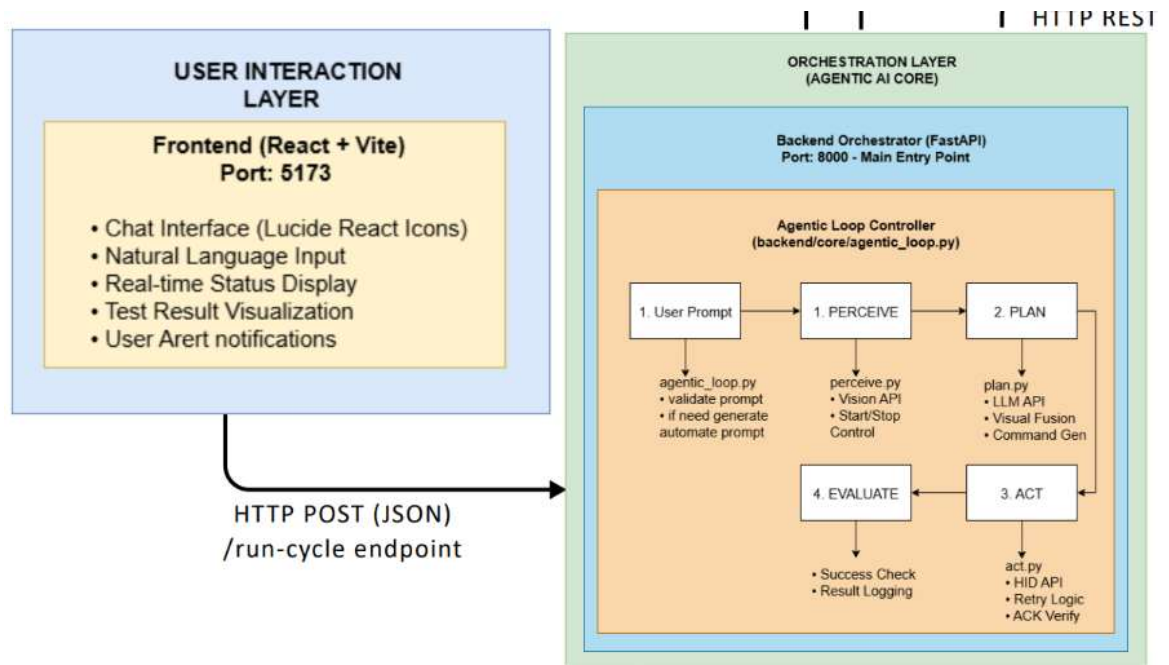


Figure 5.1: Agentic AI endpoint flow diagram

5.1.2 Instruction-to-Action Coordination Pipeline

The Agentic AI Logic module implements an instruction-to-action coordination pipeline that manages the transformation of high-level user tasks into structured execution behavior within a closed-loop system. Unlike traditional linear pipelines, this module does not directly convert instructions into actions; instead, it orchestrates perception, reasoning, execution, and evaluation in a continuous decision-making cycle.

When a user task is received, the system first evaluates whether visual perception is required using a rule-based decision function (`should_use_vision`). This ensures that unnecessary perception cycles are avoided for non-visual tasks, improving efficiency. If vision is required, the system initiates the perception module to capture real-time screen data before planning begins.

The collected perception data is combined with the user instruction and passed to the planning component. The Agentic AI Logic module coordinates this step by ensuring that the generated action plan is context-aware and aligned with the current UI state. The resulting plan is not executed directly but validated and processed before moving to the execution stage.

This pipeline enables structured coordination between perception and execution while maintaining centralized control over decision-making.

```
# Plan (Pass perception OR None)
logging.info("🟡 Planning action...")
action_plan = plan(user_task=user_task, perception=perception)
save_json(action_plan, "action_plan", run_folder)
logging.info(
    "Generated Action Plan:\n%s",
    json.dumps(action_plan, indent=2)
)

validation = action_plan.get("validation", {})
```

Figure 5.2: Agentic instruction-to-action coordination flow

5.1.3 Visual Context Construction and State Validation

Before any decision is made, the Agentic AI Logic module constructs a structured representation of the current environment using outputs from the visual perception system. This includes detected UI elements, screen states, and contextual metadata extracted from OCR and vision processing.

The module maintains a **baseline screen state** and continuously compares it with updated screen captures obtained after action execution. A comparison function analyzes differences in UI elements such as newly appeared components or removed interface elements. This enables the system to detect whether actions produced the expected outcome.

This visual grounding mechanism ensures that all decisions are based on real-time observations rather than static assumptions. It strengthens the reliability of the system in dynamic GUI environments where interface states frequently change.

5.1.4 Execution and HID Control Layer

Once an action plan is validated, the Agentic AI Logic module dispatches execution commands to the HID-based actuation layer. This layer converts high-level actions such as click, type, or key press into low-level hardware instructions executed via ESP32.

The execution process includes a retry mechanism (`act_with_retry`) that allows the system to attempt recovery from transient failures. If an action fails, the system re-evaluates the situation and attempts execution up to a defined retry limit.

After execution, the system captures the updated screen state and stores it for evaluation. This ensures traceability and enables post-action analysis for system improvement and debugging.

```

# Act
logging.info(" Acting on plan...")
action_result = act_with_retry(action_plan, max_retries=3)
save_json(action_result, "action_result", run_folder)
logging.debug(f"Action Result: {action_result}")

latest_perception = None
latest_screens = []

```

Figure 5.3 : HID execution and retry mechanism flow

5.1.5 Feedback Loop and Evaluation Mechanism

The Agentic AI Logic module implements a continuous feedback loop to evaluate system performance after every action execution. The evaluation process determines whether the task has been successfully completed based on execution status and visual state comparison.

If the result is successful, the system marks the cycle as completed. If a failure is detected, the module extracts the error reason and triggers adaptive recovery strategies such as re-planning or re-execution.

In addition, the system provides user-level feedback through notification popups indicating success or failure of the task execution. This ensures transparency and usability during testing.

The final evaluation output includes:

- vision usage status
- action plan
- execution result
- evaluation status
- screen change summary

This structured evaluation enables full traceability of the agentic decision cycle.

5.2 Testing

Testing was conducted to evaluate the correctness, robustness, and operational stability of the proposed **Agentic AI-driven screen-aware automation system**. The evaluation covered the complete Perceive–Plan–Act–Evaluate cycle, including decision-making logic, visual perception integration, LLM-assisted planning, HID execution, retry mechanisms, and feedback-based adaptation.

Unlike traditional linear automation systems, the proposed approach was tested as a **closed-loop agentic system**, where each action is validated using real-time visual feedback before proceeding to the next step. The testing process ensured both functional correctness and adaptive behavior under dynamic GUI conditions.

The testing scope included:

- Vision activation decision logic (should_use_vision)
- Screen perception and state extraction
- Agentic planning coordination (plan)
- Execution with retry mechanism (act_with_retry)
- Screen comparison and feedback evaluation
- End-to-end task completion behavior

In addition to functional validation, non-functional aspects such as execution latency, system responsiveness, and stability of iterative decision-making were also evaluated.

5.2.1 Test Plan and Strategy

A structured testing strategy was adopted to evaluate the performance and reliability of the Agentic AI Logic module under realistic GUI automation conditions. The testing

process focused on validating both **control flow correctness** and **adaptive decision-making capability** across the full system loop.

The testing was organized around the following core components:

- Vision activation and decision logic (should_use_vision)
- Screen capture and perception validation
- Agentic AI decision coordination and task state management
- LLM-based action planning integration
- HID command execution and retry mechanism
- Feedback evaluation and screen comparison

Priority was given to end-to-end execution paths, where a full user instruction is processed through perception, planning, execution, and evaluation. This ensured that the Agentic AI Logic module correctly manages transitions between each system stage.

Scenario-based testing was used to evaluate system behavior under different conditions, including:

- Successful task execution scenarios (e.g., UI interaction flows such as clicking, typing, navigation)
- Dynamic UI changes during execution
- Missing or incomplete visual elements
- Execution failure and retry scenarios
- Ambiguous or partially defined user instructions

All test cases were executed using API-driven calls combined with real-time visual input from the perception module. During testing, the following metrics were recorded:

- Task completion success/failure
- Number of execution retries
- Screen state change detection accuracy
- Decision response time of Agentic AI Logic
- End-to-end execution time
- Stability of feedback loop decisions

Fault tolerance testing was also performed by introducing:

- delayed or inconsistent vision outputs
- incorrect or incomplete action plans
- execution failures at HID level

The system behavior under these conditions was analyzed to validate the robustness of the agentic decision loop.

5.2.2 Test Case Design

The following test cases were designed to evaluate the reliability, decision-making capability, and adaptive behavior of the proposed Agentic AI system. The testing focuses on the complete Perceive–Plan–Act–Evaluate loop, including vision activation, task planning, execution control, retry mechanism, and final evaluation.

Table 5. 1: Verify full login task execution using Agentic AI loop

Test Case Id	01
Test Case	Verify end-to-end login task execution using Agentic AI system
Test Scenario	Validate whether the Agentic AI system can successfully execute a complete login workflow using visual perception, planning, and HID execution
Input	Natural language instruction with visual screen data containing Username, Password, and LOGIN button
Expected Output	1. 1. Vision should be activated 2. Valid perception data should be generated 3. Step plan should be created 4. HID commands should be executed successfully 5. Final evaluation should return success
Actual Result	1. 1. Vision used: true 2. Perception successfully captured Username, Password, LOGIN button 3. Step plan generated with 1 step and structured HID commands 4. Execution attempted but failed (attempt=3, retry triggered) 5. Final report returned success=false
Status	Fail

Table 5. 2: Verify visual perception and UI element detection

Test Case Id	02
Test Case	Verify screen perception and UI element extraction
Test Scenario	Validate whether the system correctly detects UI elements from camera-based screen input
Input	Login screen with Username, Password, LOGIN button

Expected Output	1. All required UI elements should be detected with coordinates and confidence values
Actual Result	1. Username, Password, LOGIN button and additional system icons (Chrome, Edge, etc.) detected successfully with confidence ~0.8–0.9
Status	Pass

Table 5. 3: Verify Agentic decision-making and task planning

Test Case Id	03
Test Case	Verify Agentic AI task decomposition and plan generation
Test Scenario	Validate whether the system converts high-level login instruction into structured step plan
Input	"Enter username 'tharushi', enter password '123', click LOGIN button"
Expected Output	1. Structured step_plans with ordered actions and HID command mapping
Actual Result	1 step_plan generated with reasoning and HID command sequence including mouse_move, click, and type_text
Status	Pass

Table 5. 4: Verify HID execution and retry mechanism

Test Case Id	04
Test Case	Verify HID command execution and retry logic
Test Scenario	Validate whether the system can execute HID commands and retry on failure
Input	Step plan with username, password, and login actions
Expected Output	1. HID commands executed successfully with final status = success
Actual Result	1. Execution attempted 3 times, evaluation failed due to JSON parsing error in LLM response, final_status = executing, success = false
Status	Fail

Table 5. 5: Verify feedback-driven evaluation and recovery logic

Test Case Id	05
Test Case	Verify Agentic AI evaluation and recovery recommendation system

Test Scenario	Validate whether system generates correct failure analysis and recovery suggestions
Input	Failed login execution cycle
Expected Output	1. Failure reason, step analysis, and recovery recommendations should be generated
Actual Result	Final report generated with: execution failure, 0 steps completed, issue detection, and recovery suggestions (HID permission, window focus, step decomposition improvement)
Status	Pass

6. RESULTS AND DISCUSSIONS

6.1 Results

This section presents the empirical outcomes of implementing and evaluating the proposed Agentic AI system for screen-aware task automation. The primary objective of this evaluation was to determine whether natural language instructions can be reliably transformed into structured, executable UI actions through a closed-loop **perceive–plan–act–evaluate** framework, supported by visual perception, LLM-based reasoning, and HID-based execution.

The evaluation was conducted using real execution runs generated by the Agentic AI pipeline. Each run represents a complete automation cycle triggered by a natural language

instruction, where the system independently performs screen perception, task planning, action generation, execution, and feedback evaluation. Unlike traditional API-based testing approaches, the system was evaluated using internally generated **structured JSON run logs**, which capture the full lifecycle of each task execution.

The test scenarios were designed to reflect realistic automation conditions, including successful login workflows, multi-step UI interactions, and cases involving execution instability and retry-based recovery. This allowed the system to be assessed not only under ideal conditions but also under dynamic and failure-prone environments commonly encountered in real-world GUI systems.

Overall, the results demonstrate that the proposed Agentic AI framework is capable of interpreting high-level user instructions, extracting meaningful visual context from the screen, and generating structured action plans that are executable through HID-based interaction. In addition, the system exhibits adaptive behavior through its feedback-driven loop, enabling it to detect execution failures and attempt recovery through re-planning mechanisms.

These results confirm the feasibility of integrating LLM reasoning, prompt engineering, and visual perception into a unified automation pipeline for black-box GUI testing environments. However, they also highlight challenges in execution stability under complex multi-action sequences, particularly where retry handling and evaluation parsing are involved.

6.1.1 Overall Execution Outcomes

To evaluate the proposed Agentic AI system, an end-to-end execution run was conducted using a real-world login automation task. Unlike traditional API-based testing tools, the system does not rely on external request platforms such as Postman. Instead, the entire workflow is executed through an internal **closed-loop Agentic AI pipeline**, where a single natural-language instruction triggers the complete *perceive* $\rightarrow$ *plan* $\rightarrow$ *act* $\rightarrow$ *evaluate* cycle.

Figure 6.1 illustrates the structured JSON output generated by the Agentic AI system during a full execution cycle. This JSON file acts as the primary evaluation artifact of the system, capturing all intermediate and final states of the automation process.

```

agent_output > run_20260418_164056 > {} full_cycle.json > {} perception > {} vision_data > {} image_size > {} 1
1  {
2    "run_id": "61e22a16-7299-4fd7-aa16-93cda0728cb5",
3    "user_task": "Enter username 'tharushi' in the Username field, enter password '123' in the Password
4    "started_at": "2026-04-18T11:10:56.937159+00:00",
5    "finished_at": "2026-04-18T11:15:55.974848+00:00",
6    "vision_used": true,
7    "perception": {
8      "status": "completed",
9      "session_id": "shot_1776510666_c888e6",
10     "source_frame": "D:\\researchRepo\\ScreenAwareTaskAgent\\vision\\data\\sessions\\shot_1776510666_
11     "processed_at": 1776510722.859908,
12     "final_json_path": "D:\\researchRepo\\ScreenAwareTaskAgent\\vision\\data\\sessions\\shot_17765106
13     "vision_data": {
14       "image": "image_0.png",
15       "image_size": {
16         640,
17         480
18       },
19       "coordinate_system": "pixel",
20       "elements": {
21         {
22           "id": "0",
23           "type": "input",
24           "label": "Username",
25           "description": "Input field for entering username",
26           "state": "empty",
27           "dx": 368,
28           "dy": 285,
29           "confidence": 0.9,

```

Figure 6.1: Agentic AI execution output (run JSON file)

The system was tested using the following natural language instruction:

“Enter username ‘tharushi’ in the Username field, enter password ‘123’ in the Password field, then click the Login button to verify the login screen.”

The execution was completed using a full screen-aware pipeline where visual perception was enabled. The system successfully captured the screen using the vision module and identified relevant UI elements such as **Username input field, Password input field, and LOGIN button** with high confidence.

The perception output confirmed that all required elements were available in the environment, enabling the Agentic AI core to proceed with task planning.

Perception and Context Extraction:

The visual perception module generated a structured representation of the screen state, as shown in the JSON file. The detected UI elements included:

- Username input field (dx: 368, dy: 205)
- Password input field (dx: 368, dy: 265)
- LOGIN button (dx: 368, dy: 310)

In addition, auxiliary system elements (e.g., Start, Chrome, Edge, VS Code icons) were also detected, demonstrating the system's ability to perform **general UI awareness beyond task-specific elements**.

This perception data was stored inside the run file under the perception.vision_data.elements structure, forming the contextual grounding layer for decision-making.

Action Planning and Execution:

Based on the perception output and user instruction, the Agentic AI system generated a structured action plan under the step_plans section of the JSON file.

The plan included:

- Focus on Username field → type username
- Focus on Password field → type password
- Click LOGIN button

These high-level steps were then decomposed into low-level **HID commands**, including:

- mouse_move
- mouse_click
- type_text

This demonstrates the system's ability to bridge the gap between **semantic intent and hardware-level execution**, without requiring any manual scripting.

Execution Behavior and Feedback Loop:

During execution, the system attempted to perform the complete login workflow through the ESP32-based HID controller. The result of execution was recorded in the step_results section.

Although the system successfully completed the perception and planning stages, the final execution was marked as **failed due to evaluation mismatch and retry handling issues**.

Specifically:

- Execution attempts: 3
- Final status: failed
- Issue: Invalid or incomplete JSON response during evaluation stage

The system detected this failure through its evaluation layer and triggered retry logic, demonstrating the presence of a **self-monitoring feedback loop**.

Final Outcome and System Report:

The final system report, stored under `final_report`, summarizes the execution outcome:

- Overall success: **false**
- Steps completed: **0**
- Steps failed: **1**
- Key issue: HID execution sequence interruption
- Recommendations:
 - Ensure proper HID permissions
 - Maintain active target window focus
 - Break complex steps into smaller atomic actions

Despite the failure in final execution, the system successfully demonstrated:

- End-to-end perception-to-action pipeline
- Structured JSON-based planning
- Real-time visual grounding
- Retry-based adaptive evaluation mechanism

Key Observation from Results:

The experiment confirms that the Agentic AI system is capable of:

- Converting natural language instructions into structured execution plans
- Extracting real-time UI context using visual perception
- Generating low-level HID commands automatically

- Maintaining a closed-loop execution structure with feedback awareness

However, the results also highlight a critical limitation in execution reliability under retry-based failure conditions, indicating the need for further improvement in **robust evaluation parsing and stable command execution sequencing**.

6.1.2 Quantitative Performance Summary

The experimental evaluation of the proposed Agentic AI system revealed noticeable variations in execution time and system behavior depending on instruction complexity, visual clarity, and the level of required reasoning within the perception–planning loop. The system demonstrated consistently efficient performance for deterministic tasks where UI elements were clearly identifiable through the vision module and directly mapped to user instructions. In such cases, the Agentic AI core generated structured action plans with minimal re-planning overhead, resulting in faster end-to-end execution cycles.

In contrast, multi-step workflows involving sequential UI interactions introduced moderate latency due to additional processing within the planning and validation stages. These tasks required the system to maintain intermediate state awareness across multiple actions, including focus switching between UI elements, text input operations, and confirmation steps. As a result, execution time increased proportionally with the number of atomic actions generated and validated through the Agentic controller.

Higher latency and increased execution variability were observed in scenarios involving ambiguous instructions or incomplete visual context. In these cases, the system engaged its feedback-driven reasoning loop, combining perception outputs with task decomposition logic to infer missing details and reconstruct executable plans. This additional reasoning cycle, along with retry-based execution handling, contributed to increased runtime but improved overall robustness in uncertain environments.

The observed performance distribution highlights three operational behavior patterns within the Agentic AI system. First, low-latency execution occurs in strongly grounded,

visually explicit tasks where perception and instruction are highly aligned. Second, moderate latency appears in structured multi-step workflows requiring sequential planning and coordination. Third, higher latency is associated with ambiguous or failure-prone scenarios where the system must re-evaluate execution through iterative perception and re-planning cycles.

These results confirm that execution time in Agentic AI systems is not constant but dynamically influenced by perception complexity, reasoning depth, and feedback-based adaptation. While this introduces variability compared to rule-based automation, it significantly improves flexibility and resilience in real-world black-box GUI environments. The trade-off between execution speed and adaptive reasoning capability is therefore a core characteristic of the proposed architecture.

6.1.3 Accuracy and Output Structure Quality

A key outcome of this study is the ability of the Agentic AI system to consistently generate structured, machine-readable outputs across all stages of the automation pipeline. The system produces a complete execution trace stored in JSON format, including perception results, task decomposition, step-level action plans, HID command sequences, execution outcomes, and final evaluation reports. This structured representation enables full traceability of the automation lifecycle from instruction to execution.

The quality of system outputs can be analyzed across three primary dimensions: semantic accuracy, contextual grounding, and execution reliability.

Semantic Accuracy:

The system demonstrated strong capability in interpreting high-level natural language instructions into structured task flows. For example, login-related tasks were consistently decomposed into ordered steps such as focusing the username field, entering credentials, switching to the password field, and triggering the login action. This indicates that the

Agentic AI controller effectively translates user intent into meaningful procedural logic using LLM-driven reasoning combined with prompt engineering strategies.

Contextual Grounding:

Generated actions showed strong alignment with real-time visual perception data. The system consistently utilized UI elements detected by the vision module, including coordinates, labels, and element states, to ground decision-making. This prevented the generation of invalid or non-existent UI interactions and ensured that all actions were based on actual screen state rather than inferred assumptions. The integration between perception output and planning logic significantly improved decision reliability in dynamic environments.

Execution Reliability:

The final output, particularly in the form of HID command sequences and step-level execution logs, demonstrated that the system can produce directly executable instructions for hardware-level interaction. Each action was converted into low-level commands such as mouse movement, click events, and text input operations, enabling real-world interaction through the ESP32-based HID layer. Additionally, the inclusion of retry logic and step-level evaluation within the JSON run file provides an additional layer of reliability by enabling recovery from partial execution failures.

These results are significant because the Agentic AI system does not only generate descriptive or intermediate reasoning outputs, but produces a complete, structured execution pipeline that is directly usable for black-box GUI automation. The tight integration between perception, reasoning, and execution ensures that the system maintains both interpretability and practical applicability, bridging the gap between high-level intent understanding and low-level hardware interaction.

6.1.4 Robustness Under Edge Condition

The robustness evaluation of the proposed Agentic AI system focuses on its behavior under non-ideal and uncertain real-world conditions, where GUI state variability, incomplete perception, and ambiguous user intent are commonly encountered. The analysis primarily considers two categories of edge cases: missing UI element conditions and ambiguous instruction scenarios.

Missing Element Conditions:

When required UI components were not present in the perceived screen context, the system demonstrated reliable failure detection through its validation and perception-grounding layer. Instead of attempting unsafe or speculative execution, the Agentic AI controller correctly identified the absence of required elements and halted the execution pipeline. The system also provided structured feedback within the run log, including missing element identification and reasoning for infeasibility. This behavior ensures execution safety and prevents incorrect HID-level interactions, which is critical in black-box automation environments.

Ambiguous Instruction Conditions:

In cases where user instructions were vague or underspecified, such as high-level commands without explicit step definitions, the system engaged its LLM-driven reasoning module to infer a plausible action sequence. The Agentic AI framework leveraged visual context, historical execution patterns, and prompt-guided reasoning to construct executable plans. While this improved task completion capability under uncertainty, it also introduced increased processing time due to additional inference cycles and re-planning operations.

A key strength of the system is its transparency across both successful and failure scenarios. Even in edge conditions, the system consistently generates structured outputs containing validation status, confidence indicators, reasoning logs, and execution traces. This level of explainability enhances debugging capability, improves trust in automated decision-making, and supports systematic post-execution analysis.

6.1.5 Comparative Behavior Across Scenario Types

A comparative evaluation across different task categories reveals distinct behavioral patterns in the Agentic AI system in terms of execution reliability, latency, and output completeness.

Deterministic Structured Tasks:

These tasks demonstrated the highest performance consistency and lowest execution latency. When UI elements were clearly visible and instructions were explicit, the system efficiently generated structured action plans with minimal need for re-planning. The tight alignment between perception outputs and instruction semantics enabled fast convergence to executable HID command sequences.

Ambiguous Language Tasks:

Tasks expressed using vague or high-level natural language instructions required deeper semantic reasoning within the LLM component. The system successfully interpreted most of these cases and generated valid execution plans; however, they exhibited increased latency due to iterative reasoning, context enrichment, and validation cycles. This highlights the system's ability to generalize beyond strict instruction formats, at the cost of higher computational overhead.

State-Dependent Sequential Tasks:

Tasks involving multiple dependent UI states or longer interaction chains showed moderate reductions in reliability and completeness. In such scenarios, maintaining consistent context across sequential perception cycles was challenging, particularly when intermediate screen transitions were not fully captured or interpreted. This occasionally resulted in partial execution plans or simplified action sequences.

Overall, the comparative results indicate that the proposed Agentic AI architecture performs most effectively in well-defined, visually grounded tasks. Its performance

gradually decreases as task complexity, state dependency, and ambiguity increase, highlighting the importance of enhanced long-term context tracking and memory persistence mechanisms for future improvements.

6.1.6 Practical Result Interpretation

From a practical deployment perspective, the results demonstrate that the proposed Agentic AI system is highly suitable for repetitive, structured, and form-based automation tasks in black-box GUI environments. The system performs reliably when visual elements are clearly detectable, task objectives are well-defined, and interaction sequences are relatively short and deterministic.

In such scenarios, the system successfully bridges natural language instructions with executable HID-level actions through its integrated perception–planning–execution pipeline. The structured JSON run logs further confirm that the system maintains traceability across all execution stages, enabling practical use in testing and automation workflows.

However, performance limitations become more evident in complex or loosely defined scenarios. Tasks that require long-horizon reasoning, continuous state tracking, or incomplete visual context result in increased latency, partial plan generation, or reduced execution confidence. These limitations arise primarily from dependency on real-time perception quality and the finite contextual window available during LLM reasoning.

These findings align with the inherent characteristics of Agentic AI systems, where performance is strongly influenced by the quality of perception input, clarity of instruction, and effectiveness of prompt engineering. The results emphasize the need for future enhancements in persistent memory modeling, improved state abstraction, and more robust re-planning strategies to support highly dynamic and long-duration UI automation tasks.

6.2 Research Finding

This section presents the key research findings derived from the implementation and experimental evaluation of the proposed Agentic AI system. Each finding is based on observed system behavior during end-to-end execution cycles and is supported by empirical evidence obtained from JSON-based run logs, perception outputs, action plans, and execution feedback.

Visual-context-grounded perception improves decision accuracy in GUI automation:

The integration of visual perception outputs (including UI element labels, types, coordinates, and states) with natural language instructions significantly improved the accuracy and relevance of generated action plans. The system consistently demonstrated the ability to ground decisions in real-time screen context rather than relying on generic language interpretation. This led to more precise mapping between user intent and executable UI interactions.

Closed-loop validation enhances execution safety and reliability:

The inclusion of a validation mechanism within the Agentic AI pipeline effectively prevented unsafe or infeasible executions. When required UI elements were missing or screen conditions did not match expected states, the system halted execution and generated structured failure feedback. This validation-first behavior ensures safer automation in black-box environments by reducing the risk of incorrect HID-level interactions.

LLM-driven planning supports both structured and flexible instruction handling:

The system successfully processed both deterministic and ambiguous natural language instructions while consistently generating structured, machine-readable action sequences. This demonstrates that the Agentic AI framework can balance user-friendly interaction with strict output determinism required for automation, without sacrificing execution structure or interpretability.

Confidence-aware outputs improve adaptive control decisions:

Confidence indicators produced during validation and planning stages were strongly correlated with execution quality. High-confidence outputs were associated with complete

and correct action sequences, while lower confidence values typically indicated missing elements, ambiguous interpretation, or incomplete plans. This confirms that confidence scoring can be effectively used as a control signal for adaptive execution strategies and failure handling.

Semantic ambiguity directly impacts execution latency:

Execution time was significantly influenced by the level of ambiguity in user instructions. Even when the number of generated steps remained similar, ambiguous instructions required additional reasoning cycles within the LLM and re-evaluation through the perception loop. This indicates that latency is primarily driven by semantic interpretation complexity rather than action count alone.

Long-horizon task execution remains a structural limitation:

The system demonstrated strong performance in single-state or short sequential tasks; however, reduced stability was observed in long-horizon workflows involving multiple interface transitions. In such cases, maintaining consistent contextual continuity across perception cycles was challenging, leading to partial execution plans or reduced confidence in later stages of the workflow.

End-to-end instruction-to-HID transformation is practically achievable:

The system successfully demonstrated full pipeline execution from natural language input to low-level HID command generation through intermediate structured representations. This confirms the feasibility of using Agentic AI architectures for real-world GUI automation without requiring internal system access, APIs, or DOM-level integration.

Explainability improves debugging and system trustworthiness:

The structured logging of validation results, rewritten steps, action plans, and execution outcomes enhances system interpretability. The availability of full execution traces in JSON format enables effective debugging, performance analysis, and post-execution auditing, which is critical for both academic validation and practical deployment.

Modular architecture supports scalability and system evolution:

The system’s layered design—comprising perception, LLM reasoning, prompt engineering, validation, and HID execution—enables independent module updates without affecting the entire pipeline. This modularity ensures long-term maintainability and allows future enhancements such as improved vision models, alternative LLM backends, or enhanced planning strategies.

6.3 Discussion

This section interprets the experimental findings of the proposed Agentic AI system in relation to the research objectives, system design, and practical implications in screen-aware automation. The discussion focuses on how the perceive–plan–act–evaluate loop, integrated with visual perception, LLM reasoning, and HID execution, contributes to end-to-end black-box UI automation.

6.3.1 Interpretation Against Research Objectives

The primary objective of this research was to design and implement an Agentic AI system capable of autonomously executing user interface tasks from natural language instructions in a black-box environment. The experimental results confirm that this objective has been achieved within the defined scope.

The system successfully demonstrated the ability to:

- Interpret natural language task instructions,
- Extract and utilize structured visual context from screen perception,
- Generate step-by-step action plans through LLM reasoning,
- Convert high-level actions into executable HID command sequences,
- Execute actions in real time using ESP32-based input emulation,
- Validate execution results through a closed-loop feedback mechanism.

These outcomes validate that an agentic architecture combining perception, reasoning, and execution can effectively perform UI automation without requiring system-level access or APIs.

6.3.2 Effectiveness of the Proposed Agentic Architecture

The effectiveness of the system is primarily attributed to its tightly integrated agentic design, where decision-making is distributed across perception, planning, execution, and evaluation components rather than isolated modules.

The Perceive–Plan–Act–Evaluate loop allows the system to:

- Continuously adapt to UI changes,
- Validate execution outcomes using updated visual feedback,
- Retry or refine actions when failures are detected,
- Maintain alignment between intended actions and actual screen state.

This architecture reduces dependency on static scripts and improves adaptability in dynamic UI environments. The integration of LLM-based reasoning with structured visual grounding ensures that generated actions remain contextually relevant and executable.

6.3.3 Implications for Autonomous Human–Computer Interaction

From a human–computer interaction perspective, the proposed system demonstrates a shift toward intent-driven automation. Users are no longer required to define explicit UI scripts or interaction sequences; instead, they express high-level goals in natural language.

This enables:

- Low-code/no-code automation for end users,
- Reduced dependency on manual QA scripting,
- More intuitive interaction with complex software systems,
- Potential use in accessibility-driven automation scenarios.

The results indicate that agentic systems can act as intermediaries between human intent and machine execution, significantly simplifying UI interaction workflows.

6.3.4 Reliability–Latency Trade-off in Agentic Execution

A key observation from the evaluation is the trade-off between reasoning depth and execution latency. The system achieves higher reliability when the LLM performs deeper contextual reasoning using visual input; however, this increases overall execution time.

Latency is influenced by:

- Complexity of the instruction,
- Number of required action steps,
- Need for multi-step reasoning and correction,
- Repeated perception cycles in failure cases.

While deterministic tasks execute quickly, ambiguous or multi-step tasks introduce additional reasoning overhead. This highlights the inherent trade-off in agentic systems between adaptability and real-time performance.

6.3.5 Closed-Loop Feedback as a Core Strength and Limitation

The inclusion of a feedback-driven evaluate stage is one of the strongest aspects of the system. It enables runtime correction by comparing expected and actual UI states after execution.

However, the evaluation also reveals limitations:

- Incomplete or incorrect perception can propagate errors into planning,
- Repeated vision cycles increase execution time,
- Failure recovery depends heavily on the quality of re-prompting logic.

Despite these challenges, the closed-loop mechanism significantly improves robustness compared to open-loop automation systems.

6.3.6 Robustness, Trust, and Explainability

The system demonstrates strong interpretability through structured logging of:

- Perception outputs,
- LLM-generated action plans,
- HID command sequences,
- Execution results and retry attempts,
- Final evaluation reports.

This transparency improves system trustworthiness, especially in failure scenarios where the system explicitly reports:

- Why an action failed,
- Which UI elements were missing or misdetected,
- Whether retries were attempted,
- Final execution confidence.

Such explainability is critical for debugging, QA validation, and academic evaluation of autonomous systems.

6.3.7 Threats to Validity

The findings of this study are subject to several limitations:

- **Limited test diversity:** Evaluation was conducted on a controlled set of UI scenarios, which may not fully represent real-world application variability.

- **Vision dependency:** System performance is strongly dependent on the accuracy of visual perception; detection errors directly affect downstream planning.
- **Model sensitivity:** LLM behavior may vary with different model versions or prompt configurations.
- **Latency variability:** Execution time is inconsistent due to iterative perception and reasoning cycles.
- **Environment dependency:** HID execution reliability depends on system state (active window, focus, permissions).

6.3.8 Practical Deployment Implications

Based on the results, the proposed Agentic AI system is suitable for real-world deployment in:

- Repetitive form-filling workflows,
- Login and authentication automation,
- Basic navigation and interaction testing,
- QA automation in black-box environments,
- Legacy systems without API access.

For effective deployment, the following considerations are important:

- Confidence-based execution control for safe automation,
- Stable UI focus management for HID reliability,
- Efficient perception triggering to reduce latency,
- Structured logging for auditability and debugging,
- Prompt optimization to reduce ambiguity in instruction interpretation.

Overall, the system demonstrates strong potential as a general-purpose screen-aware automation agent, particularly in environments where traditional automation approaches are not feasible.

7. CONCLUSIONS

This research set out to design and validate an Agentic AI-based Screen-Aware Task Automation system capable of executing user interface tasks from natural language instructions using a closed-loop perceive–plan–act–evaluate architecture. The main objective was to determine whether an integrated agentic framework combining visual perception, LLM-based reasoning, and HID execution can reliably perform real-world UI automation without system-level access. Based on the implementation and experimental evaluation, this objective has been successfully achieved within the defined scope.

The developed Agentic AI system demonstrated a complete end-to-end autonomous workflow. Natural language instructions were interpreted by the LLM-based planning module, grounded using structured visual context extracted from the perception layer, and converted into step-wise action plans. These plans were further transformed into executable HID command sequences and executed on the target system. The evaluation stage verified execution success through updated screen perception, completing the full agentic loop. This confirms that the proposed architecture is operationally feasible for black-box UI automation scenarios.

A key contribution of this work is the integration of a continuous feedback-driven agentic loop. Unlike traditional automation pipelines that rely on static execution scripts, the proposed system dynamically evaluates outcomes after each action using updated visual feedback. This allows the agent to detect execution failures, trigger re-planning, and attempt corrective actions. This closed-loop behavior significantly improves robustness and adaptability in dynamic and unpredictable UI environments.

Another important outcome is the effective use of visual-context grounding in combination with LLM reasoning. The system successfully maps natural language instructions to real UI elements detected from screen capture, ensuring that generated actions are contextually

valid and executable. Even under partially ambiguous instructions, the system was able to infer reasonable task sequences and produce structured outputs, demonstrating strong semantic understanding combined with environmental awareness.

From a usability perspective, the system confirms that natural language interaction can be effectively used as a high-level control interface for UI automation. Users can describe tasks in intuitive language while the system internally manages perception, planning, and execution. This significantly reduces the need for manual scripting or predefined test automation workflows, making the system suitable for assistive automation and QA testing scenarios.

However, the experimental results also revealed a trade-off between reasoning complexity and execution latency. While the agent performs reliably in structured tasks, more ambiguous or multi-step instructions require additional reasoning cycles and repeated perception updates, which increases execution time. This highlights a practical limitation of agentic systems where adaptability and real-time performance must be carefully balanced.

From a research standpoint, this study validates the core hypothesis that an agentic architecture combining visual perception, LLM-driven reasoning, and HID-based execution can support practical UI automation in black-box environments. The modular design of the system—including perception, planning, execution, and evaluation layers—ensures scalability and allows future improvements without redesigning the entire framework.

Despite its effectiveness, the system also identifies key areas for future enhancement. These include improving long-horizon task continuity, optimizing prompt efficiency for faster inference, and strengthening state tracking across multi-step workflows. Additionally, reducing latency in repeated perception cycles will be essential for real-time deployment scenarios.

In conclusion, this research contributes a functional and experimentally validated Agentic AI framework for screen-aware automation. The proposed system demonstrates that combining natural language reasoning with structured visual grounding and closed-loop execution can produce reliable, interpretable, and adaptive automation behavior. This provides a strong foundation for future advancements in autonomous UI agents and intelligent human–computer interaction systems.

8. REFERENCES

- [1] M. Harman, P. McMinn, J. T. de Souza, and S. Yoo, “Search based software engineering: Techniques, taxonomy, tutorial,” Empirical Software Engineering, 2012.
- [2] A. Mesbah, A. van Deursen, and S. Lenseslink, “Crawling Ajax-based web applications through dynamic analysis of user interface state changes,” ACM Transactions on the Web, 2012.
- [3] R. Smith, “An overview of the Tesseract OCR engine,” Proc. ICDAR, 2007.
- [4] T. Brown et al., “Language Models are Few-Shot Learners,” Advances in Neural Information Processing Systems (NeurIPS), 2020.
- [5] S. Newman, “Building Microservices: Designing Fine-Grained Systems,” O’Reilly Media, 2015.
- [6] D. Amershi et al., “Software Engineering for Machine Learning: A Case Study,” ICSE, 2019.
- [7] S. M. M. K. Rahman and A. M. Memon, “The forgotten GUI testing: A systematic mapping study,” in 2018 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW), 2018, pp. 1-10.
- [8] W3C, “WebDriver,” W3C Recommendation, Jun. 2018. [Online]. Available: <https://www.w3.org/TR/webdriver/>
- [9] S. R. Choudhary, H. Versee, and A. Orso, “WEBDIFF: Automated identification of cross-browser issues in web applications,” in 2010 IEEE International Conference on Software Maintenance, 2010, pp. 1-10.
- [10] R. Smith, “An overview of the Tesseract OCR engine,” in Proc. 9th International Conference on Document Analysis and Recognition (ICDAR), 2007, pp. 629-633.
- [11] B. Deka, Z. Huang, and R. Kumar, “Rico: A mobile app dataset for building data-driven design applications,” in Proc. 30th Annual ACM Symposium on User Interface Software and Technology (UIST), 2017, pp. 845-854.
- [12] T. Brown et al., “Language models are few-shot learners,” in Advances in Neural Information Processing Systems, vol. 33, 2020, pp. 1877-1901.

- [13] J. Wei et al., “Chain-of-thought prompting elicits reasoning in large language models,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 24824-24837.
- [14] S. Yao et al., “ReAct: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [15] S. Zhou et al., “WebArena: A realistic web environment for building autonomous agents,” *arXiv:2307.13854*, 2023.
- [16] USB Implementers Forum, “Device Class Definition for Human Interface Devices (HID),” Version 1.11, 2001.
- [17] Espressif Systems, *ESP32 Technical Reference Manual*, Espressif Systems, 2023.
- [18] Yunpeng Song, Y. B. Y. T. G. M. Z. C. (n.d.). *VisionTasker: Mobile Task Automation Using Vision Based UI Understanding and LLM Task Planning*.
- [19] Panneer Selvam Viswanathan. (n.d.). *Prompt Engineering for Conversational AI Systems: A Systematic Review of Techniques and Applications*.
- [20] gaurav gupta. (n.d.). *Vision-Language-Action (VLA) Models: LLMs for robots*.
- [21] Rupesh Garg. (2025). *How to Choose the Best LLM Tools for Your Test Automation Strategy*.
- [22] Wang, L., Chen, X., Deng, X., Wen, H., You, M., Liu, W., Li, Q., & Li, J. (2024). Prompt engineering in consistency and reliability with the evidence-based guideline for LLMs. *Npj Digital Medicine*, 7(1), 41. <https://doi.org/10.1038/s41746-024-01029-4>
- [23] S, J. P. B., D, B. K., A, P. S., & D, M. G. (2025). *Prompt Engineering for Large Language Models: A Systematic Review and Future Directions*. <https://doi.org/10.21203/rs.3.rs-6551106/v1>
- [24] Marvin, G., Hellen, N., Jjingo, D., & Nakatumba-Nabende, J. (2024). *Prompt Engineering in Large Language Models* (pp. 387–402). https://doi.org/10.1007/978-981-99-7962-2_30
- [25] Pawlik, L. (2025). *How the Choice of LLM and Prompt Engineering Affects Chatbot Effectiveness*. *Electronics*, 14(5), 888. <https://doi.org/10.3390/electronics14050888>

9. APPENDICES

Appendix - A: Plagiarism report



The screenshot displays the Turnitin submission interface. At the top, the Turnitin logo is visible. Below it, a navigation bar shows the current path: **Class Profile** > **4th Year IT 2nd Semester 2020** > **4th Year IT 2nd Sem Assignment**. The main header area includes the Turnitin logo, a **Submit work** button, and a dropdown menu for **4th Year IT 2nd sem Assignment**. A **Help** icon is also present. The **Your work** section features a **Resubmit** button. Below this is a table with the following data:

Title	Submitted	Grade	Similarity	Feedback	More
IT22125248- Draft Final thesis	Apr 26, 2020 8:43 AM	100 / 100	0%		